27/03/26 — ISW - Teórico

01

Gestión de configuración de Software

- SCM (Software Configuration management) → Importante para mantener la Integridad y orden de Software — Satisfacer al usuario / Trazabilidad
  - Software: Programas, Procedimientos, reglas, Documentación, Datos. Tiene varias representaciones. A medida que transcurre el tiempo, cambia / Evoluciona → Errores, cambios en negocio / Requerimientos / Productos Asociados
  - Disciplina de Soporte; transversal al Proyecto
  - Control de Calidad de Procesos y Productos; Pruebas de software ~~y~~ → Administración de config. de software (SCM)
  - Definición: Dirección y apoyo técnico a Identificar y Documentar características funcionales
  - Ítem de configuración: cualquier artefacto que puede cambiar la configuración y sea parte del Producto o del Proyecto, que puede ser compartido entre los miembros del equipo → Variante: ítem alternativo; Versión: ítem en un momento
  - Repositorio: Contenedor de (IC), que puede ser uno o varios BD. Centralizados / Descentralizados
  - Líneas Base: configuración previamente revisada y formalmente que puede generar a desarrollos posteriores. No es Versión de Producto. Permite reproducir el entorno de desarrollo en un momento dado y se ha definido formalmente
  - Definición: en gestión tradicional de Proyecto: Identificar ítems de configuración, controlar los cambios, registrar y reportarlos junto con su estado y verificar correspondencia con los requerimientos
  - Ramas: Líneas alternativas; pueden revertirse a la rama principal; bifurcan el desarrollo
  - Identificación de Ítems de configuración: - Identificación unívoca de cada ítem - convenciones y reglas de nombrado
    - Elementos: - De Producto: EBS, Arquitectura, Definir estructura del repositorio - ubicación dentro del repo (ciclo manual) - De Proyecto: Plan de Proyecto, cronograma - De Iteración: Plan de Iteración, reporte de defectos
  - Control de cambios: Se origina con un requerimiento de cambio → 1 ó N (IC), en una línea base. Es un procedimiento formal que involucra actores y una evaluación del impacto del cambio
  - Auditorías de Configuración de Software: Validación y Verificación, Auditoría Funcional y Física
    - Física: Asegura que lo indicado para cada IC esté realmente documentado
    - Funcional: Evaluación de los Productos para que cada IC sea consistente con la EBS
  - Informes de Estado: Se mantienen los registros de la evolución del sistema; Se puede implementar dentro de Procesos automáticos

Proyecto / Producto

Conjunto = Config. de Software

ELEMENTOS PRINCIPALES

Actividades fundamentales

SCM

- GCS en Ágil: sirve al equipo de desarrollo y lo contiene. Responde a los cambios, busca ser transparente con un feedback continuo.
  - Es responsabilidad de todo el equipo, debe ser educado

02

31/03/26 — ISW - Teórico

- Filosofía Requerimientos ágiles

Se valora más tener:
- Individuos e Interacción sobre Procesos y Herramientas
- Software funcionando sobre Documentación extensiva
- Colaborar con el cliente sobre Negociación contractual
- Responder el cambio sobre seguir un Plan

- Qué es Ágil?: No es una metodología o Proceso, es una ideología que guían el desarrollo del Producto con un conjunto definido de Principios → Planificación continua; Facilitadores - Equipos cortados; Integración continua, Prácticas de Ingeniería disciplinadas.
  - La diferencia inmediata es la existencia de una menor cantidad de Documentación, sin embargo no es esto lo más importante, sino ser adaptables en lugar de Predictivos y ser orientados al cliente en lugar del Proceso. No significa entregar Productos incompletos, sino Software funcionando que va evolucionando. Entregando el Producto desde el camino más efectivo
  - Requerimientos en Ágile: usan el valor para construir el Producto correcto → usar historias y modelos para mostrar que construir → Determinar que es "sólo lo suficiente". Un Producto "exitoso" tiene un alto Porcentaje de desperdicio.
  - Los Requerimientos se ordenan por orden de Prioridad, pueden ser cambiantes y los Primeros aportan valor de Negocio.
  - Tipos de Requerimientos: Dentro del Problema: - De Negocio - De usuario; Dentro de la Solución: - Funcional - No Funcional - Implementación
  - En un Alcance de una Iteración se administran ciertos requerimientos; en Ágil, brindar valor; en cuenta tiempo y recursos, incluyo tal feature; Tiempo igual → Tradicional: tiempo diferentes

En síntesis: Se busca entender la necesidad y negocio → Se entrega valor frecuentemente a los stakeholders
→ Descubriendo la solución de forma colaborativa → Junto a un equipo competente

- Principios ágiles Relacionados a requerimientos ágiles:
  1- Prioridad: Satisfacer al cliente a través de releases tempranos y frecuentes
  2- Recibir cambios de requerimientos Aun en etapas finales
  3- Técnicos y no técnicos trabajando Juntos todo el Proyecto
  4- El medio de comunicación por excelencia es cara a cara
  5- Los mejores Arquitecturas, diseños y requerimientos emergen de equipos Autoorganizados

Leer No Silver Bullet

→ Foco en usuario

* User stories: Como <Nombre Rol>, Yo quiero Puedo <actividad>, de forma que <Valor de Negocio Recibido>
  Son: - Una necesidad del usuario - Una descripción del Producto - Un ítem de Planificación
  - token de conversación - Mecanismo para detener una conversación
  - El Product owner Prioriza las stories - Las stories Impactan en muchas áreas del Producto (GUI, Business Logic, database)

Tienen Criterios de Aceptación

- Modelo de Rol: Tarjeta de Rol, • Pruebas de Aceptación: Proponer posibles casos de uso de user
- INVEST Model: - Independent: Calendarizables e Implementables en cualquier orden
  - Negotiable: El "qué" no el "cómo"
  - Valuable: Debe tener valor para el cliente
  - Estimable: Para ayudar al cliente a tomar un ranking basado en costos
  - Small: Deben ser concretos en una iteración
  - Testable: Demostrar que fueron Implementadas

Importante

- Definition of Done
- Check-List de user stories

- Funcionales cuando hay Interacción entre de Gran interdependencia de usuario con el sistema, mejor si es un Prototipo

• Técnicas: Investigación enfoques en torno de la solución

- user stories → Theme → Epic
- Spikes: Sirve para investigación, de incertidumbre de operaciones para user stories. Son técnicas y funcionales Estimables, Demostrables y Aceptables. Las spikes son para incógnitas críticas

10/04/26 ISW - Teórico

03 * Estimaciones ágiles en Ingeniería de Software
- Axioma Pragmático: Aceptación de incertidumbre entrica, utilizando procesos de trabajo iterativos para adaptarnos a realidad cambiante, las estimaciones no son absolutas, son relativas, la combinación de muchas estimaciones crean una "estimación absoluta", con sesgos asumidos

La incertidumbre se estima, se toma

- Estimación: Herramienta utilizada para realizar un Plan, Sirve como entrada
  - No son compromisos - "Posible respuesta tentativa" sobre la factibilidad de un trabajo
  - Lo beneficioso es el Proceso de hacerlos - Puede servir como una gran Protección para el equipo

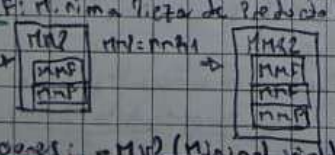
El nuevo Paradigma propone realizar mediciones relativas, comparando A" B según velocidad, colaboración, Eficiencia

- Story Point: Medida de un equipo sobre ~~una User Story~~ Features/Stories que posee una serie de variables Atentar
  Variables: - Complejidad: Dificultad técnica → crece exponencialmente - Esfuerzo: Volumen de trabajo bruto - Dudas: Nivel de incertidumbre
  Debe concentrarse en equipo → Experto, aprovechamiento del conocimiento técnico del equipo
  → Analogía: comparación entrada
  → - Cónica: User story pequeña → Desagregación nutritiva de Archivos / tareas
  → Se debe de realizar un conjunto de puntaje en tarjetas → Estimación
- Aprende en base de las iteraciones previas; Estimar en Story Points permite una estimación más real, solamente al tiempo y esfuerzo

04 * Gestión de Producto:
- Arquitectura del Valor en Productos de Software: Desde la hipótesis de que el Producto le da valor al cliente se plantea la hipótesis. Gran parte de las features no se usan. Pasar de las tareas de un Producto a la experiencia de usuario es una de las tareas más difíciles. "Audacia del cero": Preferible tener clientes reales

Hipótesis Desarrollar

- MVP: Experimento para ver si la feature es necesaria o no, o Aportara valor al cliente. El objetivo es encontrar la feature que generará valor común entre todos. Un MVP tiene una hipótesis, para ver si el mercado está interesado
- MMF: Mínima Pieza de Producto que puede Salir a Vender - MVF: Mínima nueva función que yo puedo agregar



- MLP: Cual es el Producto mínimo que el usuario estará satisfecho
- MVF (Minimal viable Feature): Como un MVP a menor escala, Preferentemente a usuarios ya existentes

Iterativo

• Relaciones: - MVP (Minimal viable Product): una nueva versión del Producto, creado con el menor esfuerzo posible, utilizado para obtener aprendizaje validado. Es más cercano a un Prototipo

Mucho valor

- MMF (Minimal Marketable Feature): Pieza de Funcionalidad más pequeña que puede ser lanzada

Ya Pasa

- MMP (Minimal Marketable Product): Primer Realease de un MMR, siendo más "cerca" accesible

Conjunto de features

- MMR (Minimal Marketable Realease): Lanzamiento de un Producto que tiene el conjunto de características más pequeño posible

skycolor

17/04/26 — ISW - Teórico

* Componentes de Proyecto de Desarrollo de Software

- Arquitectura del Proyecto:
  - Entradas: Requerimientos, Ideas, Tiempo
  - Procesador: Actividades
  - Salidas: Productos y Servicios (de Software)
  - Pilares del Motor del Proyecto: Personas (Habilidades, Entrenamiento, Motivación), Herramientas (Equipos, Tecnologías), Proceso (Secuencia de Pasos), Producto (Bienes o Servicios)

Proceso empírico (Definido) — Adaptarse a los cambios del desarrollo

- Ajuste parte a un proceso / Proyecto Atrasado lo Atrasa más

- Definido vs Empírico
  - Línea de Ensamblaje — Caminos Adaptativos
  - Variables: Estáticas — Variables cambiantes y estadísticas
  - Predictibilidad: Alta — Predictibilidad: Baja
    - repetir genera mismo resultado — Repetición genera diferentes resultados
  - Control: Guiado por — Control: Adaptación Continua

- Tipos de ciclos de vida: (secuencial)
  - Iterativo
  - Secuencial
  - Recursivo
  - Estimación anticipada de usar tiempo

- Proceso: Conjunto de Reglas, actividades y pasos que se siguen para desarrollar y mantener el Software
- Proyecto: un esfuerzo temporal con inicio y fin donde Aplica Procesos, Secuencias, y Estimaciones
- Producto: lo que se vendera y Entrega Resultado del Proyecto

- Anatomía de un Proyecto:
  - Duración Limitada: Temporario. Cuando se alcanza el objetivo se termina
  - Orientado a objetivos: Guiado por resultados claros, Específicos Alcanzables
  - Únicos: Tareas interdependientes basadas en esfuerzos y recursos específicos que no se han hecho Antes

- en Proyecto: Alcance, Esfuerzo, Tiempo (Calendario), Recursos, Costos

- Ciclo de vida: La serie de pasos a través de los cuales el Producto o Proyecto Refresca
  - De Producto: Idea → Plan de Negocio → Producto → Operaciones / Actualización → Retiro
  - De Proyecto: Fase Inicial → Fase Intermedia → Fase Final (Retrocesos)

- Medición de Éxito: Producto: Medido contra Requerimientos, todas las características incluidas
  Proyecto: Cuando todas las Fases culminaron
- En una buena Gestión de Proyecto, Proyectos de alta calidad entregan el Producto requerido, en tiempo estimado con Presupuesto Planificado.
- Variables de control: Alcance — Tiempo — Costo; Es Responsabilidad del Líder de Proyecto balancear estos 3 factores en conjunto (Triple Restricción)

- El Plan:
  - Alcance: Características del Producto vs trabajo a Realizar para que el Producto tenga todos los Requerimientos
  - Ciclo de vida: Iterativo, Secuencial, Incremental
  - Estimaciones: cantidad calculada; cuanto se va a tardar, costar o requerir (que, cuando, cómo y quien)
  - Puestos: Arquitectos y otra
  - Métricas: Instrumentos de Control → Medidas similares
- A medida que el Proyecto avanza las sobre y sub estimaciones distribuyen conforme nos acercamos a las estimaciones y resultados reales.
- un error en las estimaciones es la de omitir actividades; como Actividades de Desarrollo: como Documentación, testing, Mantenimiento de versiones, Pruebas, revisión de código. Actividades Generales: como Licencias, Días de Enfermedad, Capacitaciones, Reuniones

- Formas de Estimación:
  - Basadas en Experiencia:
  - " " Analogía
  - " " Descomposición

→ un riesgo es un Posible Problema esperando en suceder, se Identifican Amenazas, se Analiza su Impacto, Planificamos estrategias de mitigación, luego de Seguimiento constante

→ Dentro de las métricas se divide en: De Proceso — De Proyecto — De Producto (Identificar métricas)
(Tamaño de Proceso, Esfuerzo, Tiempo, Defectos)

- Causas Comunes de Fracaso:
  Fallas al Definir el Problema Planificar basado en Datos Insuficientes, no hay Seguimiento del Plan, Nadie estaba a Cargo, Las estimaciones se basaron en Supuestos

* Criterio de Aceptación: Debe estar registrado y valorado. Controlar los controles

24/04/26 — Ejal - Práctico/teórico

* Múltiples niveles de Planificación: - Estrategia - Portfolio - Producto - Release - Iteración - Día

N → Forma de trabajo de los releases

↳ Varios productos ↳ Varios productos

- Portfolio: que invertir
- Producto: cómo evolucionar
- Release: qué y cuándo entregar
- Sprint: qué hacemos ahora
- Día: cómo avanzamos

- Estimaciones: Se vuelven más precisas a medida se bajan el nivel
- Planificación de Estrategia: Nivel más alto de Planificación. Define hacia donde va la organización. Depende del mercado, cliente, valor, y posicionamiento

Niveles de Planificación

- Planificación de Portfolio: Define qué productos o iniciativas invertir. Busca maximizar el valor de negocio
- " de Producto: Traduce la Estrategia en una visión concreta del producto. Es una solución concreta que genera valor al usuario. Participan PO, Scrum. Se parte de una idea inicial y se definen recursos, y restricciones. Se genera visión, backlog personalizado y roadmap. Permite ordenar entregas futuras
- " de Release: Define qué features se entregarán. Inputs: backlog, visión, velocidad del equipo. Outputs: Plan de release (qué y cuándo) Refinamos las user stories y product backlog
- Cadencia de sprint: Define cada cuanto se entrega valor. Más frecuencia = Mayor feedback
- Planificación de Iteración (Sprint Planning): Se define la primera actividad de un sprint
- Planificación Diaria: Reunión breve. Sirve para sincronizar el progreso y el avance del plan. Permite detectar bloqueos

- Estimaciones con un Entrada

Justificar

- Esfuerzo: Bajo - Medio - Alto - Nulo 3 - 5 - 8 - 13
- Complejidad:
- Incertidumbre:
- Definir historias y user stories: Dos ventas de prendas

H: Existe gente interesada en un sitio de ventas de prendas de segunda

- Cómo conocer quien busca un tipo de prenda para comprar eficientemente.

Necesita o sist.

Roles: vendedor, encargado de logística, Analista Selección, Analista Publicación, Admin → No sive

la compradora → No sive Ex: Por qué usa el Mark y que no

Del lado del usuario

- Listado de U.S.

(AS) (AP) (N)

- Publicar prenda - Enviar formulario - Seleccionar prenda - Generar propuesta de valor - Confirma propuesta de venta
- Registrar usuario

Desarrollar una Entera. Criterios de Aceptación. Frase verbal. Propósito de valor?

06

* Estimaciones de SW: una estimación no es precisa; son la base de los planes pero estos no tienen que ser lo mismo que lo estimado. A mayor diferencia entre lo estimado y planeado, hay mayor riesgo.

- Métodos: - Basados en experiencias en datos históricos previos
  - " Juicio Experto: un experto estima las estimaciones y hace la suya. Se basa enteramente en el esfuerzo del experto. Es 75% es de efectivo. Más rápido a la práctica
  - " Delphi: Estimaciones en grupo

skycolor

00 * Introducción a Ing. de Software: en general, es un set de Programas y la documentación que acompaña. hay 3 tipos de software: De sistema, utilitarios, y de Aplicación. Entre las razones para no considerar Software y manufactura tenemos: - El Software es menos Predecible - No hay Producción en masa, casi ningún Producto de Software es igual a otro. - No todos los fallos son errores - El Sw no se gasta y no está gobernado por Leyes de la Física

- Errores en Desarrollo de SW: - La Versión final no satisface las necesidades del cliente - No es fácil adaptarlo o Expandirlo - Mala documentación - Mala calidad - Más tiempo y costo que Presupuestado
- Medidas Influyentes en el Desempeño
  - Positivas (mejor o menor): - Involucramiento de usuario - Apoyo de la gerencia - Requerimientos CLAROS - Planteamiento adecuado - Expectativas realistas - Hitos intermedios
  - Negativas (mayor o menor): - Requerimientos incompletos - Falta de involucramiento al usuario - Falta de recursos - Expectativas poco realistas - Falta de apoyo de la gerencia - Requerimientos cambiantes
- El Proceso de Software: Conjunto de actividades a desarrollar para la creación de un sist. de software. Estas actividades varían dependiendo de la organización y el tipo de sistema que debe desarrollarse. Debe ser explícitamente modelado si va a ser administrado
- Proceso Definido vs Empírico: Los Procesos Definidos son aquellos caracterizados por su nivel de Predictibilidad Alto, como su nivel de control y administración. Podemos repetir indefinidamente el Proceso y obtener los mismos resultados. Por otro lado, los Procesos empíricos asumen Procesos complejos, con variables cambiantes; podemos repetirlo y obtener resultados diferentes cada vez. La administración y control eficiente de los mismos se basa en una constante inspección y adaptación a los cambios Presentes. Estos Procesos trabajan bien en Procedimientos creativos y complejos
- un ciclo de vida de un Proyecto de software, es una representación de un Proceso. Grafica una descripción del Proceso desde una Perspectiva Particular. Los Modelos especifican las fases del Proceso y su orden

* Problemas de Software (No sirve Bullet): Se dividen en dos categorías basadas en conceptos involucrados
- Dificultades Esenciales: Son inherentes a la naturaleza del Software, como diseño de Algoritmos complejos y estructuras de Datos. La Parte más difícil es la especificación y diseño de esos constructos.
- Dificultades Accidentales: Surgen de la representación del SW en lenguajes o entornos específicos. Se identifican 4 Características que hacen que sean difícil de construir:
- Características
  - Complejidad: Las construcciones de SW son más difíciles que cualquier otra porque es difícil su repetición, y su complejidad crece no linealmente con su tamaño
  - Conformidad: El SW debe adaptarse arbitrariamente a interfaces diseñados por humanos
  - Maleabilidad: El SW está bajo constante Presión de cambio Porque es fácil de modificar, y su función es la Parte que más evoluciona de un sistema
  - Invisibilidad: No tiene una realidad Física. Esto impide que existe menos visualizar herramientas Poderosas
- En el software no existen las leyes de Plata, es decir, aunque hay una herramienta que permita o facilite totalmente el desarrollo del mismo. Algunas herramientas que tienen considerados leyes de Plata son:
  - Lenguajes orientados de Programación - POO - IA - Prog. Automática
- En lugar de soluciones rígidas se Propone adoptar Pori - Conjunto de Requerimientos Probados - Desarrollo de Requisitos y Prototipado Rápido (Desarrollo Iterativo Incremental) - Cultivar Fronteras Diseñadas

28/04/26

ISW - Práctico

Enviar formulario de Prendas a Vender → User: tiene Valor Para el Usuario - Científica: Justificación; Podemos citar en MVP → Justificación Verbal Informal

Si se cumple... debe ...

Como VENDEDOR quiero completar y enviar un formulario con mis Prendas PARA que el equipo de logística contacte el retiro o entrega de las mismas / Ver si puedo Venderlas online.

Decisiones técnicas/funcionales

- Estimación: 5. Baja complejidad, es una Plantilla, no requiere mucho esfuerzo y aún así Parece incierto, sólo está de elegir el servicio para enviar mails
- Criterio de Aceptación: - Se indica cant. de Prendas por categoría, descripción y foto (512 kb) y forma de envío (Puede)
  - El Vendedor debe estar logueado X
  - Se debe poder al menos la Prendas, contener email
- Pruebas de usuario: - Se suben la Prendas con categoría, descripción, foto (512 kb), la forma de envío. El Vendedor está logueado y recibe el mail (Fase)
  - Se cargan 5 Prendas sin categoría, sin descripción, sin foto, sin adjuntar forma de envío. El vendedor no tiene vertrado el número de teléfono (No Fase)

En base a los criterios

Se Prueba Subir

No Probar

Probar que Funcionen con ...(Fallo)

Detalles Varios Pruebas de Falla

skycolor

02/05/26 — ISW - Teórico

07 * SCRUM

- Pilares del empirismo - Transparencia - Inspección - Adaptación

→ Framework Agile: Es un marco de trabajo liviano que ayuda a las personas, equipos y organizaciones a generar valor a través de soluciones adaptativas para problemas complejos

* Elementos de Scrum

- Artefactos: - Product Backlog: objetivo del producto (DoD - Definition of Ready)
  - Sprint Backlog: " " Sprint
  - Product Increment: DoD - (Definition of Done)
- Responsabilidades: - Scrum master - Product owner - Developers
- Eventos: - Sprint: 1 mes Max
  - " Planning: 8 hs máximo
  - Daily meeting: 15 min
  - Sprint Review: 4 hs por mes
  - " Retrospective: 3 hs Max
  - Refinamiento - 10%

→ Valor de negocio claramente expresado, Detalles confirmados por el equipo, Dependencias identificadas, es PSI que es estimado y puede ser completado en un sprint, los criterios de aceptación (Acceptance) son claros y testeables.

- DoD: - Diseño Revisado - Código completo - Doc Actualizada - Proyecto (unidad / integración / sistema)
  - No hay defectos - Pruebas de Aceptación ok - En servers de Producción

- Métricas en ágile: Regla de oro: La medición es una salida, no una actividad. Medir solo lo necesario
  - La mayor prioridad es satisfacer al cliente por medio de entregas tempranas y continuas de software valioso
  - El software trabajando es la principal medida de progreso
- Velocity: Métrica del progreso de un equipo. Se calcula sumando número de story points asignados a cada user story que el equipo completa en la iteración. Solo se suman las stories completas